

Tile-Level Communication for Fused Multi-GPU Inference

Make the cross-GPU *communication schedule* a first-class, demand-aware library concern:

declare → **schedule** → **execute**

Subha

AMD · advised by Simran Arora & Muhammad Osama
TileComm · 8×MI350X (gfx950, all-to-all XGMI) · built on HipKittens + IRIS

July 7, 2026

Going a level up: from a fused region to a comm abstraction

Last time I showed the fused DeepSeek-R1 MoE region. Today I go one level up: make the *communication schedule itself* a first-class, demand-aware library concern — the tile-level abstraction Awad and Osama pointed at — and I close out the MXFP4 and memory questions.

Where I was

Fused MoE region.
1.56× prefill over the
MORI → aiter → MORI
baseline. HipKittens
compute + IRIS comm.

Where I'm going

The comm abstraction. A
tile-granular collective the
library *schedules* — the
paper Awad + Osama
pointed at.

Hardware

8×MI350X, gfx950. All-
to-all XGMI. DeepSeek-
R1 MXFP4, config
TP4×DP2 + DP-
attention + EP.

A fused kernel declares which tiles go to which ranks; the library picks the link-balanced schedule — instead of every kernel hand-rolling its own.

Recap — the fused region works, and it told me where to look

- ▶ *Prefill*: $1.56\times$ over the unfused MORI-dispatch $\rightarrow$ aiter-fmoe $\rightarrow$ MORI-combine baseline (1247 vs 1941 μ s), correctness-gated.
- ▶ We fuse the expert *region*: the gather *produces the GEMM's layout*, so *7 kernels collapse to 3* — no sort, no standalone quant, no scatter-combine.
- ▶ *Decode is a weight wall* — the GEMM streams all 32 local experts' ~ 1.4 GB of fp8 weights every step; fusion caps at $\sim 1.25\times$. So fp4 + the comm work matter more than more GEMM tuning.

The thread I'm pulling on

Our biggest single communication win didn't come from a better *algorithm*. It came from *changing the order* in which thread blocks issue their remote writes.

That's a *scheduling decision* — and right now it's hand-rolled. Making it a library concern is the contribution.

The observation — $2.4\times$ from one hand-rolled line

Same kernel, same bytes, same reduction. The *only* change: how destination rows are ordered across the 8 XGMI links.

combine schedule	region μs	why
<i>sorted by dst rank</i> (natural CSR order)	<i>934 μs</i>	a window of concurrent blocks hammers <i>one</i> link
<i>round-robin cells</i> (interleave=True)	<i>386 μs</i>	spreads the window across all 8 links
delta	$2.42\times$	and now <i>beats</i> MORI EpCombine (386 vs 398)

The hook

`irisx/fused_moe/example.py` $\rightarrow$ `build_combine_pull(interleave=True)` — a human noticed the link imbalance and hand-wrote a round-robin. *That instinct is worth $2.4\times$. The question is why a human had to have it.*

The problem — that round-robin is right by luck

workload-blind

Balances count, not bytes.

Under the expert imbalance Simran flagged — some experts get $8\times$ the tokens — a hot expert's tile is far bigger, and cycling ranks equally leaves links unbalanced.

not reusable

Re-derived per collective.

The gather has its own ordering; a future `reduce_scatter` needs a third. Every collective re-invents link balancing from scratch, with its own bugs.

a cliff, not a slope

The default is pathological.

The order you get for free from a CSR grouped by destination is the $934\mu\text{s}$ one. Miss the hand-tuning and you're $2.4\times$ slow — silently.

The schedule should be computed by the library from the declared demand and the measured link topology — not authored by hand, per kernel, once per collective.

TileComm — declare the transfer, let the library schedule & execute

1 · declare

What moves. A set of tile transfers {tile, src, dst, size, op?} — not load/store in a hand-chosen order.

2 · schedule

When & over which link. The library link-balances the order from demand + probed XGMI topology — *the traffic-shaper.*

3 · execute

How it runs. Lowers to IRIS primitives inside a HipKittens kernel — bulk-synchronous now, tile-fused later.

```
tile_gather(tiles, src_map) # dispatch: pull
    from owners
tile_scatter(tiles, dst_map) # push to
    destinations
tile_reduce_scatter(tiles, dst,sum) # combine,
    all-reduce/2
tile_all_reduce(tiles, sum) # rs + all-gather
four intents cover the MoE region + the TP collectives
```

The point

The author writes the declaration *once*. Bulk and fused executions are two lowerings of the *same* intent, and the schedule is shared. The permutation space Osama called “unexplored” — how many tiles before you reduce — becomes a schedule the library *searches*, not a constant a human tunes.

The quadrant — tile-granular + demand-adaptive + fused is empty

	whole-tensor	tile-granular
fixed algorithm	NCCL / RCCL — ring, tree; no fusion	—
programmable	MSCCL / MSCCL++ — buffer-level, hand-authored	<i>TileComm</i>
fused into a kernel	aiter / MORI — schedule baked in	<i>TileComm</i>

- ▶ *IRIS* gives device primitives (load/store/get/put/atomic) + one whole-tensor `all_reduce`. Tile collectives don't exist — so we hand-roll them.
- ▶ *HipKittens* gives a gorgeous *tile compute* abstraction (register/shared tiles, MFMA) and *no communication concept at all*.

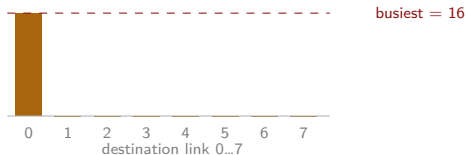
The missing verb

HipKittens describes *tiles of compute*. IRIS moves *bytes between ranks*. Nothing describes *tiles of communication* that a scheduler can balance and a GEMM can overlap.

Why we *think* order matters — and an honest caveat

A window of 16 concurrent remote writes over 8 XGMI links. *Makespan is set by the busiest link* (the tallest bar): sorted piles the window onto one link; round-robin spreads it.

sorted: one link saturated



round-robin: all 8 links busy



Caveat (honest)

The on-node probe I ran to test this was *issue-bound* — IRIS stores are fire-and-forget, so it timed *issue rate*, not link bandwidth (implied ~ 4.7 TB/s, $>10\times$ the fabric) and showed only $\sim 1.05\times$. So the mechanism is *not yet validated* in isolation. The $2.4\times$ is a real *region* result; whether it's link-spreading or the combine's read/reduction structure is still being isolated. Fixing the probe's completion fence is next.

The cost model — calibrated, and an honest read of where it wins

A wave-based link-contention model (`tilecomm/tilesched.py`), calibrated so uniform combine reproduces the measured 934 / 386 μ s. Three schedulers: *sorted* (naive), round-robin (today), *proportional* (byte-weighted, demand-aware).

A · uniform combine (a *fit*, not a prediction)

schedule	model μ s	measured
<i>sorted</i>	927.8	934
round-robin	388.2	386
<i>proportional</i>	388.3	(= rr)

B · imbalanced gather — makespan vs expert skew

skew (max/mean)	round-robin	<i>proportional</i> (= LB)
1.00	780.4	780.4
1.37	1065.8	1031.5
1.94	1441.3	1411.5
3.94	2696.7	2659.8

2.4–6×

scheduling vs the naive order a kernel author writes by default — the big lever, now impossible to miss.

= lower bound

proportional stays optimal across every skew; round-robin drifts 1–4% and grows with imbalance.

honest cap

reorder alone is bounded by the hottest link — the *unbounded* win is comm/compute overlap.

Grounded — every intent already exists hand-rolled in the kernel

today, hand-rolled in <code>fused_moe/</code>	TileComm intent	the hard-coded schedule
<code>combine_pull (interleave=True)</code>	<i>tile_reduce_scatter</i>	literally <code>sched_round_robin</code>
<code>gather_pack</code> + multi-source route	<i>tile_gather</i>	implicit expert-major order
future GEMM + all-reduce (<i>Osama's prize</i>)	<i>tile_reduce_scatter</i>	<i>the schedule space to search</i>

The concrete first step is a drop-in

Replace the `interleave=True` argument with `tilecomm.schedule(transfer_set, topology)`. Regression-gate: reproduce $386\mu\text{s}$ on uniform traffic. Then beat round-robin on a real load-imbalanced routing trace. *Same combine_pull_kernel* — low risk, directly measured against a number we already trust.

MXFP4 — memory is a non-issue; the blocker is a legitimate baseline

“Will R1 fit on 8 GPUs?” — checked

- ▶ MI350X = *309 GB* HBM each → *2.47 TB* across 8.
- ▶ R1 MXFP4 (403 GB) = *16% of HBM*; fp8 (~671 GB) = 27%. Fits with huge headroom.
- ▶ The old wall was *disk*, not HBM — 666 GB free on /data2, model already cached, live SGLang-MXFP4 container on the node.

status of the fp4 kernel work

- ▶ *Route-1 W4A16 decode done & shipped* — MXFP4 weights, hardware `cvt_scalef32_pk_bf16_fp4` unpack: *1.6× GEMM*, 1.11× region over fp8. Weight-only fp4 is the *right* decode choice (decode is weight-bound).
- ▶ The honest gap: our fp4 comparison replayed a *buggy, untuned* captured aiter kernel. Beating it means nothing.

The plan — measure against the real stack, in the realistic config

Run AMD’s reproducible *SGLang DeepSeek-R1-FP4* benchmark (already on this node), profile the MoE region under *TP4×DP2 + DP-attention + EP* — *not* TP8, which inflates the all-reduce — and iterate against that trace. Simran offered to run the e2e: the gold denominator. Rule: never iterate against a kernel we captured ourselves.

Roadmap & asks — from a hand-rolled line to a tile-fused collective

1. *Fix the on-node XGMI probe* — tonight's run was issue-bound; add a completion fence + back-pressure so it tests the mechanism. Control link BW = 47 GiB/s.
2. *Drop-in into combine* — `tilecomm.schedule()` replaces `interleave`; gate at $386\mu\text{s}$, win on an imbalanced trace.
3. *reduce_scatter as a real IRIS collective* — the TP-decomposition stepping stone (bulk-synchronous first).
4. *Tile-fused execution* — emit the reduce-scatter *inside* the GEMM tile loop. The unbounded comm/compute-overlap win — Osama's prize.
5. *Multi-path routing under skew* — relay a hot flow through an idle rank. Ordering-only $\rightarrow$ full traffic engineering.

What I'd like from you

- ▶ *Simran* — the per-expert token-count routing trace, and the e2e SGLang-R1-FP4 run as the gold baseline.
- ▶ *Osama* — a gut check on the tile-fused `reduce_scatter` as the “tiles-before-reduce” schedule-search target.
- ▶ *Awad* — is *declare / schedule / execute* the right shape, and is the drop-in the right first milestone?

Honest limits: reorder-only buys $2.4\text{--}6\times$ over the naive order but $\sim 1\text{--}4\%$ over the good hand-roll; the big win is Step 4. The cost model rests on one measured point until Step 1 lands.

Backup — anticipated questions (1/3)

- ▶ *“Is 1–4% over round-robin worth a whole abstraction?”* The 1–4% isn’t the pitch — the pitch is *2.4–6× over the naive order made automatic and correct-by-construction*, robustness to imbalance for free, reuse across gather/combine/reduce_scatter, and the substrate for the tile-fused overlap win. Reorder-only is the *floor* of the value, not the ceiling.
- ▶ *“Why not just always use round-robin?”* It balances *count*, not *bytes* — under imbalance it drifts, and it can’t express the tile-fused schedule at all. And “always round-robin” is itself a per-kernel human decision the naive path doesn’t give you.
- ▶ *“How is this different from MSCCL / MSCCL++?”* MSCCL programs collectives at *buffer* granularity with *hand-authored* schedules, standalone. TileComm is *tile-granular*, *demand-adaptive* (the schedule is *computed*, not written), and designed to *fuse into the compute kernel’s tile loop*. The fusion + auto-scheduling is the delta.

Backup — anticipated questions (2/3)

- ▶ *“Your cost model is calibrated to one point — is it real?”* Upfront: it's a fit to one measured region number (combine, 934/386); the mechanism isn't validated yet. The on-node XGMI probe was *issue-bound* (IRIS stores are fire-and-forget $\Rightarrow$ measured issue rate, implying ~ 4.7 TB/s, $>10\times$ the fabric) and showed $\sim 1.05\times$. So it's *inconclusive, not a refutation*. It's a design-space tool calibrated to a real number; fixing the completion fence is the immediate next step.
- ▶ *“Does scheduling even matter, if the probe shows $\sim 1\times$?”* The probe couldn't test it (issue-bound). The $2.4\times$ is a real region measurement; what's open is *why* — pure link-spreading, or the combine's read/reduction structure. Either way the case holds: scheduling should be *empirical and library-owned* precisely because the mechanism behind a hand-tuned win isn't obvious. And the big prize (tile-fused overlap) doesn't depend on this at all.
- ▶ *“On an all-to-all fabric, can ordering beat the hottest link?”* No — reorder-only is *capped by the hottest link's bytes* (slide 8). Multi-path routing (roadmap step 5) moves that floor, and only when the demand matrix is skewed onto specific links. The unbounded win before that is hiding comm under compute (tile-fusion).

Backup — anticipated questions (3/3)

- ▶ “Does the fp4 GEMM help decode?” The tensor cores get faster but you still *stream all the experts’ weights*, so decode barely moves — *weight-only fp4 halves the weight bytes*, the actual decode bottleneck, so that’s the lever, and it’s the Route-1 choice.
- ▶ “Did you get a new end-to-end MXFP4 speedup?” No — *deliberately*. Route-1 is shipped; the missing piece is a *legitimate external baseline*, which needs the real SGLang serve. I didn’t brute-force that on a shared node tonight; the design + cost model was the higher-value use of the time, and the e2e is the clean way to get the real number.

Numbers at a glance

combine: *386 μ s* pull (round-robin) vs 934 μ s (sorted) vs 398 μ s MORI · scheduling lever *2.4–6 $\times$* over naive, $\sim$ 1–4% over the hand-roll · probe $\sim$ 1.05 $\times$ (issue-bound) · R1 MXFP4 = 403 GB = 16% of 2.47 TB · Route-1 fp4 GEMM *1.6 $\times$* , 1.11 $\times$ region · code: [irisx/tilecomm/](#)